

UPI Payment and Digital Wallet System using Interface and Exception Handling

1) Objective / Aim

To design and implement a Digital Wallet system in Java using an interface and exception handling, simulating real-world digital payment operations such as wallet creation, adding money, making UPI payments, checking the wallet balance, and validating transactions.

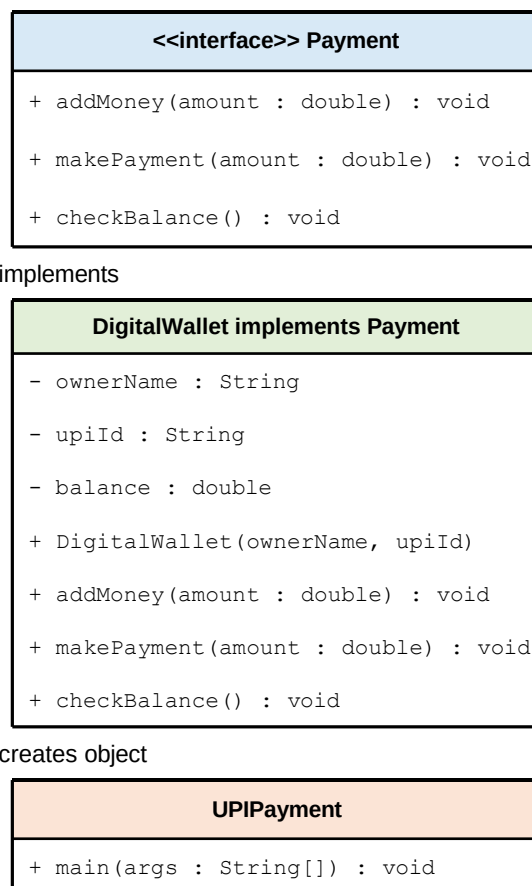
2) Execution Steps

1. Open a Java IDE or command prompt/terminal.
2. Create a source file named **UPIPayment.java**.
3. Define the **Payment** interface with methods for adding money, making payments, and checking balance.
4. Create the **DigitalWallet** class and implement the **Payment** interface.
5. Initialize the owner's name, UPI ID, and wallet balance using the constructor.
6. Use **addMoney()** to add money and handle invalid amounts using **IllegalArgumentException**.
7. Use **makePayment()** to process a payment and handle insufficient balance using exception handling.
8. Use **checkBalance()** to display the UPI ID, owner name, and current balance.
9. In the main method, take owner name, UPI ID, amount to add, and payment amount from the user.
10. Compile using **javac UPIPayment.java** and execute using **java UPIPayment**.

3) Algorithm

1. Start the program.
2. Create a **Payment** interface containing **addMoney()**, **makePayment()**, and **checkBalance()**.
3. Create the **DigitalWallet** class that implements the **Payment** interface.
4. Declare owner name, UPI ID, and balance as private data members.
5. Initialize the wallet balance to zero in the constructor.
6. In **addMoney()**, check whether the amount is greater than zero. If not, throw an **IllegalArgumentException**; otherwise add it to the balance.
7. In **makePayment()**, check whether the payment amount is valid and whether sufficient balance is available.
8. If the payment amount is greater than the balance, throw an exception and display a payment failure message.
9. If sufficient balance exists, subtract the payment amount and display the successful payment and remaining balance.
10. In **checkBalance()**, display the wallet details.
11. In **main()**, read the owner name and UPI ID, create the wallet object, add money, make a UPI payment, and display the final wallet details.
12. Stop the program.

4) UML Class Diagram



Relationship: DigitalWallet implements the Payment interface. UPIPayment contains the main method that creates and operates on the DigitalWallet object.

5) Source Code

The source code below is reproduced from the uploaded **UPIPayment.java** file and is included as selectable text.

```
import java.util.Scanner;

// Interface
interface Payment {
    void addMoney(double amount);
    void makePayment(double amount);
    void checkBalance();
}

// Digital Wallet class implements the interface
class DigitalWallet implements Payment {

    private String ownerName;
    private String upiId;
    private double balance;

    // Constructor
    DigitalWallet(String ownerName, String upiId) {
        this.ownerName = ownerName;
        this.upiId = upiId;
        this.balance = 0;
    }

    // Add money to wallet
    public void addMoney(double amount) {
        try {
            if (amount <= 0) {
                throw new IllegalArgumentException("Amount must be greater than 0");
            }

            balance = balance + amount;

            System.out.println("Money Added: Rs." + amount);
            System.out.println("New Balance: Rs." + balance);
        }
        catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }

    // Make UPI payment
    public void makePayment(double amount) {
        try {
            if (amount <= 0) {
                throw new IllegalArgumentException(
                    "Payment amount must be greater than 0"
                );
            }

            if (amount > balance) {
                throw new Exception("Insufficient wallet balance");
            }

            balance = balance - amount;

            System.out.println("Payment Successful!");
            System.out.println("Paid: Rs." + amount);
            System.out.println("Remaining Balance: Rs." + balance);
        }
        catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (Exception e) {
            System.out.println("Payment Failed: " + e.getMessage());
        }
    }

    // Check wallet balance
    public void checkBalance() {
        System.out.println("UPI ID: " + upiId);
        System.out.println("Owner Name: " + ownerName);
        System.out.println("Current Balance: Rs." + balance);
    }
}

// Main class
public class UPIPayment {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}
```

```

System.out.print("Enter Owner Name: ");
String name = sc.nextLine();

System.out.print("Enter UPI ID: ");
String upi = sc.nextLine();

// Creating wallet object
DigitalWallet wallet = new DigitalWallet(name, upi);

System.out.println("\n--- Digital Wallet Created ---");

// Add money
System.out.print("Enter amount to add: ");
double addAmount = sc.nextDouble();

wallet.addMoney(addAmount);

// Make payment
System.out.print("\nEnter amount to pay using UPI: ");
double paymentAmount = sc.nextDouble();

wallet.makePayment(paymentAmount);

// Display final balance
System.out.println("\n--- Wallet Details ---");
wallet.checkBalance();

sc.close();
}
}

```

6) Output

Two terminal outputs are included: one demonstrates a failed payment due to insufficient wallet balance, and the other demonstrates a successful UPI payment.

```
Enter Owner Name: Rahul
Enter UPI ID: rahul@upi

--- Digital Wallet Created ---
Enter amount to add: 2000
Money Added: Rs.2000.0
New Balance: Rs.2000.0

Enter amount to pay using UPI: 3000
Payment Failed: Insufficient wallet balance

--- Wallet Details ---
UPI ID: rahul@upi
Owner Name: Rahul
Current Balance: Rs.2000.0
```

```
Enter Owner Name: John
Enter UPI ID: john@upi

--- Digital Wallet Created ---
Enter amount to add: 5000
Money Added: Rs.5000.0
New Balance: Rs.5000.0

Enter amount to pay using UPI: 2000
Payment Successful!
Paid: Rs.2000.0
Remaining Balance: Rs.3000.0

--- Wallet Details ---
UPI ID: john@upi
Owner Name: John
Current Balance: Rs.3000.0
```

7) Result

The UPI Payment and Digital Wallet System was successfully implemented in Java using an interface and exception handling. The program successfully demonstrates wallet creation, adding money, UPI payment processing, balance checking, and handling of invalid or insufficient transactions.

Key Concepts Demonstrated

Concept	Demonstration
Interface	Payment interface defines common payment operations
Implementation	DigitalWallet implements Payment
Exception Handling	IllegalArgumentException and Exception are handled
Encapsulation	Wallet details are stored in private fields
Constructor	Initializes owner name, UPI ID, and balance
User Input	Scanner accepts wallet and payment details